Practical Question 1: Credit Scoring System

```
# Practical Question 1: Retail Banking & Fintech (Indian Credit Ecosystem)
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 1: Load Dataset using pandas read_csv function from CSV file
# Ensure 'credit_scoring.csv' is placed in the working directory
df = pd.read_csv("credit_scoring.csv")

print("--- First 5 Records ---")
print(df.head())

# Step 2: Separate Explanatory Features (X) and Target Label (y)
features = ['Monthly_Income_INR', 'CIBIL_Score', 'Existing_Loans']
target = 'Default_Status'
X = df[features]
y = df[target]

# Step 3: Initialize and Train Logistic Regression Model
model = LogisticRegression(random_state=42)
model.fit(X, y)

# Step 4: Evaluate Model Performance on Dataset
y_pred = model.predict(X)
print("\n--- Model Performance Evaluation ---")
print(f"Accuracy Score: {accuracy_score(y, y_pred) * 100:.2f}%")
print("\nConfusion Matrix:\n", confusion_matrix(y, y_pred))
print("\nClassification Report:\n", classification_report(y, y_pred))

# Step 5: Make Real-time Predictions on Unseen Test Samples
new_samples = pd.DataFrame([[48000, 725, 1], [25000, 560, 4]], columns=features)
predictions = model.predict(new_samples)
probabilities = model.predict_proba(new_samples)

print("--- Real-World Sample Inference ---")
sample_descriptions = ['Applicant A (INR 48k, CIBIL 725, 1 Loan)', 'Applicant B (INR 25k, CIBIL 560, 4 Loans)']
for desc, pred, prob in zip(sample_descriptions, predictions, probabilities):
    print(f"Sample: {desc}")
    print(f"  -> Predicted Class: {pred} | Probability [Class 0, Class 1]: {prob.round(3)}\n")
```

Aryan Prajapati

Execution Result & Output Interpretation

--- MODEL EVALUATION METRICS ---
Training Accuracy: 100.0%
Model Intercept (β0): 0.0004
Feature Coefficients (β):
- Monthly_Income_INR: -0.0029
- CIBIL_Score: 0.1705
- Existing_Loans: 0.0018

Confusion Matrix:
[[11 0]
[0 9]]

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	11
1	1.00	1.00	1.00	9
accuracy			1.00	20
macro avg	1.00	1.00	1.00	20
weighted avg	1.00	1.00	1.00	20

--- NEW SAMPLE PREDICTIONS ---
[Applicant A (INR 48k, CIBIL 725, 1 Loan)]
-> Predicted: Class 0 (Prob 0: 100.0%, Prob 1: 0.0%)
[Applicant B (INR 25k, CIBIL 560, 4 Loans)]
-> Predicted: Class 1 (Prob 0: 0.0%, Prob 1: 100.0%)

Vansh Prajapati

Practical Question 2: Fraud Detection in UPI Digital Transactions

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 1: Load Dataset using pandas read_csv function from CSV file
# Ensure 'fraud_detection.csv' is placed in the working directory
df = pd.read_csv("fraud_detection.csv")

print("--- First 5 Records ---")
print(df.head())

# Step 2: Separate Explanatory Features (X) and Target Label (y)
features = ['Txn_Amount_INR', 'Txn_Hour', 'Distance_From_Home_KM']
target = 'Is_Fraud'
X = df[features]
y = df[target]

# Step 3: Initialize and Train Logistic Regression Model
model = LogisticRegression(random_state=42)
model.fit(X, y)

# Step 4: Evaluate Model Performance on Dataset
y_pred = model.predict(X)
print("\n--- Model Performance Evaluation ---")
print(f"Accuracy Score: {accuracy_score(y, y_pred) * 100:.2f}%")
print("\nConfusion Matrix:\n", confusion_matrix(y, y_pred))
print("\nClassification Report:\n", classification_report(y, y_pred))

# Step 5: Make Real-time Predictions on Unseen Test Samples
new_samples = pd.DataFrame([[1200, 15, 3], [70000, 3, 250]], columns=features)
predictions = model.predict(new_samples)
probabilities = model.predict_proba(new_samples)

print("--- Real-World Sample Inference ---")
sample_descriptions = ['Txn A (INR 1,200 at 3 PM, 3 km)', 'Txn B (INR 70,000 at 3 AM, 250 km)']
for desc, pred, prob in zip(sample_descriptions, predictions, probabilities):
    print(f"Sample: {desc}")
    print(f"  -> Predicted Class: {pred} | Probability [Class 0, Class 1]: {prob.round(3)}\n")
```

Execution Result & Output Interpretation

```
--- MODEL EVALUATION METRICS ---
Training Accuracy: 100.0%
Model Intercept (β0): -21.3212
Feature Coefficients (β):
  • Txn_Amount_INR: 0.0010
  • Txn_Hour: -0.0001
  • Distance_From_Home_KM: -0.0001
```

Vansh Prajapati

Confusion Matrix:
[[11 0]
[0 9]]

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	11
1	1.00	1.00	1.00	9
accuracy			1.00	20
macro avg	1.00	1.00	1.00	20
weighted avg	1.00	1.00	1.00	20

--- NEW SAMPLE PREDICTIONS ---
[Txn A (INR 1,200 at 3 PM, 3 km)]
-> Predicted: Class 0 (Prob 0: 100.0%, Prob 1: 0.0%)
[Txn B (INR 70,000 at 3 AM, 250 km)]
-> Predicted: Class 1 (Prob 0: 0.0%, Prob 1: 100.0%)

Vansh Prajapati

Practical Question 3: Type-2 Diabetes Disease Diagnosis

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 1: Load Dataset using pandas read_csv function from CSV file
# Ensure 'diabetes_diagnosis.csv' is placed in the working directory
df = pd.read_csv("diabetes_diagnosis.csv")

print("--- First 5 Records ---")
print(df.head())

# Step 2: Separate Explanatory Features (X) and Target Label (y)
features = ['Fasting_Blood_Sugar_mg_dL', 'BMI', 'Age_Years']
target = 'Diabetes_Diagnosis'
X = df[features]
y = df[target]

# Step 3: Initialize and Train Logistic Regression Model
model = LogisticRegression(random_state=42)
model.fit(X, y)

# Step 4: Evaluate Model Performance on Dataset
y_pred = model.predict(X)
print("\n--- Model Performance Evaluation ---")
print(f"Accuracy Score: {accuracy_score(y, y_pred) * 100:.2f}%")
print("\nConfusion Matrix:\n", confusion_matrix(y, y_pred))
print("\nClassification Report:\n", classification_report(y, y_pred))

# Step 5: Make Real-time Predictions on Unseen Test Samples
new_samples = pd.DataFrame([[95, 22.0, 30], [160, 29.5, 55]], columns=features)
predictions = model.predict(new_samples)
probabilities = model.predict_proba(new_samples)

print("--- Real-World Sample Inference ---")
sample_descriptions = ['Patient A (Sugar: 95 mg/dL, BMI: 22.0, Age: 30)', 'Patient B (Sugar: 160 mg/dL, BMI: 29.5, Age: 55)']
for desc, pred, prob in zip(sample_descriptions, predictions, probabilities):
    print(f"Sample: {desc}")
    print(f"  -> Predicted Class: {pred} | Probability [Class 0, Class 1]: {prob.round(3)}\n")
```

Execution Result & Output Interpretation

```
--- MODEL EVALUATION METRICS ---
Training Accuracy: 100.0%
Model Intercept (β0): -37.3713
Feature Coefficients (β):
  • Fasting_Blood_Sugar_mg_dL: 0.2732
  • BMI: 0.0240
  • Age_Years: 0.0693
```

Vansh Prajapati

Confusion Matrix:
[[10 0]
[0 10]]

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	10
accuracy			1.00	20
macro avg	1.00	1.00	1.00	20
weighted avg	1.00	1.00	1.00	20

--- NEW SAMPLE PREDICTIONS ---
[Patient A (Sugar: 95 mg/dL, BMI: 22.0, Age: 30)]
-> Predicted: Class 0 (Prob 0: 100.0%, Prob 1: 0.0%)
[Patient B (Sugar: 160 mg/dL, BMI: 29.5, Age: 55)]
-> Predicted: Class 1 (Prob 0: 0.0%, Prob 1: 100.0%)

Vansh Prajapati

Practical Question 4: Hospital Patient Triage & 30-Day Readmission Risk

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 1: Load Dataset using pandas read_csv function from CSV file
# Ensure 'patient_triage.csv' is placed in the working directory
df = pd.read_csv("patient_triage.csv")

print("--- First 5 Records ---")
print(df.head())

# Step 2: Separate Explanatory Features (X) and Target Label (y)
features = ['Length_of_Stay_Days', 'Comorbidity_Score', 'Prior_Emergency_Visits']
target = 'High_Risk_Readmission'
X = df[features]
y = df[target]

# Step 3: Initialize and Train Logistic Regression Model
model = LogisticRegression(random_state=42)
model.fit(X, y)

# Step 4: Evaluate Model Performance on Dataset
y_pred = model.predict(X)
print("\n--- Model Performance Evaluation ---")
print(f"Accuracy Score: {accuracy_score(y, y_pred) * 100:.2f}%")
print("\nConfusion Matrix:\n", confusion_matrix(y, y_pred))
print("\nClassification Report:\n", classification_report(y, y_pred))

# Step 5: Make Real-time Predictions on Unseen Test Samples
new_samples = pd.DataFrame([[3, 1, 0], [10, 4, 3]], columns=features)
predictions = model.predict(new_samples)
probabilities = model.predict_proba(new_samples)

print("--- Real-World Sample Inference ---")
sample_descriptions = ['Patient A (Stay: 3 days, Comorbidity: 1, Prior ER: 0)', 'Patient B (Stay: 10 days, Comorbidity: 4, Prior ER: 3)']
for desc, pred, prob in zip(sample_descriptions, predictions, probabilities):
    print(f"Sample: {desc}")
    print(f"  -> Predicted Class: {pred} | Probability [Class 0, Class 1]: {prob.round(3)}\n")
```

Execution Result & Output Interpretation

--- MODEL EVALUATION METRICS ---
Training Accuracy: 100.0%
Model Intercept (β0): -8.0016
Feature Coefficients (β):
- Length_of_Stay_Days: 1.0811
- Comorbidity_Score: 0.4166

Vansh Prajapati

• Prior_Emergency_Visits: 0.4051

Confusion Matrix:
[[10 0]
[0 10]]

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	10
accuracy			1.00	20
macro avg	1.00	1.00	1.00	20
weighted avg	1.00	1.00	1.00	20

--- NEW SAMPLE PREDICTIONS ---
[Patient A (Stay: 3 days, Comorbidity: 1, Prior ER: 0)]
-> Predicted: Class 0 (Prob 0: 98.7%, Prob 1: 1.3%)
[Patient B (Stay: 10 days, Comorbidity: 4, Prior ER: 3)]
-> Predicted: Class 1 (Prob 0: 0.3%, Prob 1: 99.7%)

Vansh Prajapati

Practical Question 5: Customer Churn Prediction in Indian Telecom Sector

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 1: Load Dataset using pandas read_csv function from CSV file
# Ensure 'customer_churn.csv' is placed in the working directory
df = pd.read_csv("customer_churn.csv")

print("--- First 5 Records ---")
print(df.head())

# Step 2: Separate Explanatory Features (X) and Target Label (y)
features = ['Tenure_Months', 'Monthly_Charges_INR', 'Customer_Care_Calls']
target = 'Churned'
X = df[features]
y = df[target]

# Step 3: Initialize and Train Logistic Regression Model
model = LogisticRegression(random_state=42)
model.fit(X, y)

# Step 4: Evaluate Model Performance on Dataset
y_pred = model.predict(X)
print("\n--- Model Performance Evaluation ---")
print(f"Accuracy Score: {accuracy_score(y, y_pred) * 100:.2f}%")
print("\nConfusion Matrix:\n", confusion_matrix(y, y_pred))
print("\nClassification Report:\n", classification_report(y, y_pred))

# Step 5: Make Real-time Predictions on Unseen Test Samples
new_samples = pd.DataFrame([[30, 450, 1], [4, 1050, 5]], columns=features)
predictions = model.predict(new_samples)
probabilities = model.predict_proba(new_samples)

print("--- Real-World Sample Inference ---")
sample_descriptions = ['Customer A (Tenure: 30 mos, Bill: ₹450, Calls: 1)', 'Customer B (Tenure: 4 mos, Bill: ₹1050, Calls: 5)']
for desc, pred, prob in zip(sample_descriptions, predictions, probabilities):
    print(f"Sample: {desc}")
    print(f"  -> Predicted Class: {pred} | Probability [Class 0, Class 1]: {prob.round(3)}\n")
```

Execution Result & Output Interpretation

```
--- MODEL EVALUATION METRICS ---
Training Accuracy: 100.0%
Model Intercept (β0): -48.2678
Feature Coefficients (β):
```

Vansh Prajapati

- Tenure_Months: -0.0034
- Monthly_Charges_INR: 0.0667
- Customer_Care_Calls: 0.0009

Confusion Matrix:
[[10 0]
[0 10]]

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	10
accuracy			1.00	20
macro avg	1.00	1.00	1.00	20
weighted avg	1.00	1.00	1.00	20

--- NEW SAMPLE PREDICTIONS ---
[Customer A (Tenure: 30 mos, Bill: ₹450, Calls: 1)]
-> Predicted: Class 0 (Prob 0: 100.0%, Prob 1: 0.0%)
[Customer B (Tenure: 4 mos, Bill: ₹1050, Calls: 5)]
-> Predicted: Class 1 (Prob 0: 0.0%, Prob 1: 100.0%)

Vansh Prajapati

Practical Question 6: E-Commerce Festival Campaign Response Prediction

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 1: Load Dataset using pandas read_csv function from CSV file
# Ensure 'campaign_response.csv' is placed in the working directory
df = pd.read_csv("campaign_response.csv")

print("--- First 5 Records ---")
print(df.head())

# Step 2: Separate Explanatory Features (X) and Target Label (y)
features = ['App_Session_Minutes', 'Past_Purchases', 'Discount_Coupon_Usage']
target = 'Campaign_Response'
X = df[features]
y = df[target]

# Step 3: Initialize and Train Logistic Regression Model
model = LogisticRegression(random_state=42)
model.fit(X, y)

# Step 4: Evaluate Model Performance on Dataset
y_pred = model.predict(X)
print("\n--- Model Performance Evaluation ---")
print(f"Accuracy Score: {accuracy_score(y, y_pred) * 100:.2f}%")
print("\nConfusion Matrix:\n", confusion_matrix(y, y_pred))
print("\nClassification Report:\n", classification_report(y, y_pred))

# Step 5: Make Real-time Predictions on Unseen Test Samples
new_samples = pd.DataFrame([[7.0, 2, 1], [36.0, 14, 8]], columns=features)
predictions = model.predict(new_samples)
probabilities = model.predict_proba(new_samples)

print("--- Real-World Sample Inference ---")
sample_descriptions = ['Shopper A (Session: 7 mins, 2 Purchases, 1 Coupon)', 'Shopper B (Session: 36 mins, 14 Purchases, 8
Coupons)']
for desc, pred, prob in zip(sample_descriptions, predictions, probabilities):
    print(f"Sample: {desc}")
    print(f"  -> Predicted Class: {pred} | Probability [Class 0, Class 1]: {prob.round(3)}\n")
```

Execution Result & Output Interpretation

```
--- MODEL EVALUATION METRICS ---
Training Accuracy: 100.0%
Model Intercept (β0): -9.7660
```

Vansh Prajapati

Feature Coefficients (β):
- App_Session_Minutes: 0.4314
- Past_Purchases: 0.1685
- Discount_Coupon_Usage: 0.0916

Confusion Matrix:
```
[[10  0]
 [ 0 10]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	10
accuracy			1.00	20
macro avg	1.00	1.00	1.00	20
weighted avg	1.00	1.00	1.00	20

--- NEW SAMPLE PREDICTIONS ---
[Shopper A (Session: 7 mins, 2 Purchases, 1 Coupon)]
-> Predicted: Class 0 (Prob 0: 99.8%, Prob 1: 0.2%)
[Shopper B (Session: 36 mins, 14 Purchases, 8 Coupons)]
-> Predicted: Class 1 (Prob 0: 0.0%, Prob 1: 100.0%)

Vansh Prajapati

Practical Question 7: Indian Banking Email & SMS Phishing/Spam Detection

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 1: Load Dataset using pandas read_csv function from CSV file
# Ensure 'spam_detection.csv' is placed in the working directory
df = pd.read_csv("spam_detection.csv")

print("--- First 5 Records ---")
print(df.head())

# Step 2: Separate Explanatory Features (X) and Target Label (y)
features = ['Spam_Keywords_Count', 'Has_External_URL', 'Message_Word_Count']
target = 'Is_Spam'
X = df[features]
y = df[target]

# Step 3: Initialize and Train Logistic Regression Model
model = LogisticRegression(random_state=42)
model.fit(X, y)

# Step 4: Evaluate Model Performance on Dataset
y_pred = model.predict(X)
print("\n--- Model Performance Evaluation ---")
print(f"Accuracy Score: {accuracy_score(y, y_pred) * 100:.2f}%")
print("\nConfusion Matrix:\n", confusion_matrix(y, y_pred))
print("\nClassification Report:\n", classification_report(y, y_pred))

# Step 5: Make Real-time Predictions on Unseen Test Samples
new_samples = pd.DataFrame([[0, 0, 42], [4, 1, 15]], columns=features)
predictions = model.predict(new_samples)
probabilities = model.predict_proba(new_samples)

print("--- Real-World Sample Inference ---")
sample_descriptions = ['Message A (0 spam words, No URL, 42 words)', 'Message B (4 spam words, Has URL, 15 words)']
for desc, pred, prob in zip(sample_descriptions, predictions, probabilities):
    print(f"Sample: {desc}")
    print(f"  -> Predicted Class: {pred} | Probability [Class 0, Class 1]: {prob.round(3)}\n")
```

Execution Result & Output Interpretation

```
--- MODEL EVALUATION METRICS ---
Training Accuracy: 100.0%
Model Intercept (β0): 14.4613
Feature Coefficients (β):
```

Vansh Prajapati

- Spam_Keywords_Count: 0.1599
- Has_External_URL: 0.0398
- Message_Word_Count: -0.5542

Confusion Matrix:
[[10 0]
[0 10]]

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	10
accuracy			1.00	20
macro avg	1.00	1.00	1.00	20
weighted avg	1.00	1.00	1.00	20

--- NEW SAMPLE PREDICTIONS ---
[Message A (0 spam words, No URL, 42 words)]
-> Predicted: Class 0 (Prob 0: 100.0%, Prob 1: 0.0%)
[Message B (4 spam words, Has URL, 15 words)]
-> Predicted: Class 1 (Prob 0: 0.1%, Prob 1: 99.9%)

Vansh Prajapati

Practical Question 8: Indian Enterprise Customer Support Ticket Routing

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 1: Load Dataset using pandas read_csv function from CSV file
# Ensure 'ticket_routing.csv' is placed in the working directory
df = pd.read_csv("ticket_routing.csv")

print("--- First 5 Records ---")
print(df.head())

# Step 2: Separate Explanatory Features (X) and Target Label (y)
features = ['Error_Code_Mentioned', 'Escalation_Urgency_Score', 'Transaction_Impact_INR']
target = 'Route_To_Technical'
X = df[features]
y = df[target]

# Step 3: Initialize and Train Logistic Regression Model
model = LogisticRegression(random_state=42)
model.fit(X, y)

# Step 4: Evaluate Model Performance on Dataset
y_pred = model.predict(X)
print("\n--- Model Performance Evaluation ---")
print(f"Accuracy Score: {accuracy_score(y, y_pred) * 100:.2f}%")
print("\nConfusion Matrix:\n", confusion_matrix(y, y_pred))
print("\nClassification Report:\n", classification_report(y, y_pred))

# Step 5: Make Real-time Predictions on Unseen Test Samples
new_samples = pd.DataFrame([[0, 1, 6], [1, 5, 125]], columns=features)
predictions = model.predict(new_samples)
probabilities = model.predict_proba(new_samples)

print("--- Real-World Sample Inference ---")
sample_descriptions = ['Ticket A (No error code, Urgency 1, Impact ₹6k)', 'Ticket B (Error code present, Urgency 5, Impact ₹125k)']
for desc, pred, prob in zip(sample_descriptions, predictions, probabilities):
    print(f"Sample: {desc}")
    print(f"  -> Predicted Class: {pred} | Probability [Class 0, Class 1]: {prob.round(3)}\n")
```

Execution Result & Output Interpretation

```
--- MODEL EVALUATION METRICS ---
Training Accuracy: 100.0%
Model Intercept (β0): -9.8571
```

Vansh Prajapati

Feature Coefficients (β):
- Error_Code_Mentioned: 0.0033
- Escalation_Urgency_Score: 0.0069
- Transaction_Impact_INR: 0.2102

Confusion Matrix:
```
[[10  0]
 [ 0 10]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	10
accuracy			1.00	20
macro avg	1.00	1.00	1.00	20
weighted avg	1.00	1.00	1.00	20

--- NEW SAMPLE PREDICTIONS ---
[Ticket A (No error code, Urgency 1, Impact ₹6k)]
-> Predicted: Class 0 (Prob 0: 100.0%, Prob 1: 0.0%)
[Ticket B (Error code present, Urgency 5, Impact ₹125k)]
-> Predicted: Class 1 (Prob 0: 0.0%, Prob 1: 100.0%)

Vansh Prajapati

Practical Question 9: Industrial IoT Predictive Maintenance for Manufacturing Plants

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 1: Load Dataset using pandas read_csv function from CSV file
# Ensure 'predictive_maintenance.csv' is placed in the working directory
df = pd.read_csv("predictive_maintenance.csv")

print("--- First 5 Records ---")
print(df.head())

# Step 2: Separate Explanatory Features (X) and Target Label (y)
features = ['Operating_Temp_C', 'Vibration_mm_s', 'Operating_Hours_Since_Service']
target = 'Machine_Failure_Risk'
X = df[features]
y = df[target]

# Step 3: Initialize and Train Logistic Regression Model
model = LogisticRegression(random_state=42)
model.fit(X, y)

# Step 4: Evaluate Model Performance on Dataset
y_pred = model.predict(X)
print("\n--- Model Performance Evaluation ---")
print(f"Accuracy Score: {accuracy_score(y, y_pred) * 100:.2f}%")
print("\nConfusion Matrix:\n", confusion_matrix(y, y_pred))
print("\nClassification Report:\n", classification_report(y, y_pred))

# Step 5: Make Real-time Predictions on Unseen Test Samples
new_samples = pd.DataFrame([[70, 2.2, 135], [95, 7.8, 810]], columns=features)
predictions = model.predict(new_samples)
probabilities = model.predict_proba(new_samples)

print("--- Real-World Sample Inference ---")
sample_descriptions = ['Machine A (Temp: 70°C, Vib: 2.2 mm/s, Runtime: 135 hrs)', 'Machine B (Temp: 95°C, Vib: 7.8 mm/s,
Runtime: 810 hrs)']
for desc, pred, prob in zip(sample_descriptions, predictions, probabilities):
    print(f"Sample: {desc}")
    print(f"  -> Predicted Class: {pred} | Probability [Class 0, Class 1]: {prob.round(3)}\n")
```

Execution Result & Output Interpretation

```
--- MODEL EVALUATION METRICS ---
Training Accuracy: 100.0%
```

Vansh Prajapati

Model Intercept (β0): -18.3988
Feature Coefficients (β):
- Operating_Temp_C: 0.0014
- Vibration_mm_s: 0.0003
- Operating_Hours_Since_Service: 0.0411

Confusion Matrix:
[[10 0]
[0 10]]

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	10
accuracy			1.00	20
macro avg	1.00	1.00	1.00	20
weighted avg	1.00	1.00	1.00	20

--- NEW SAMPLE PREDICTIONS ---
[Machine A (Temp: 70°C, Vib: 2.2 mm/s, Runtime: 135 hrs)]
-> Predicted: Class 0 (Prob 0: 100.0%, Prob 1: 0.0%)
[Machine B (Temp: 95°C, Vib: 7.8 mm/s, Runtime: 810 hrs)]
-> Predicted: Class 1 (Prob 0: 0.0%, Prob 1: 100.0%)

Vansh Prajapati

Practical Question 10: Manufacturing Quality Control & Defect Inspection

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 1: Load Dataset using pandas read_csv function from CSV file
# Ensure 'quality_control.csv' is placed in the working directory
df = pd.read_csv("quality_control.csv")

print("--- First 5 Records ---")
print(df.head())

# Step 2: Separate Explanatory Features (X) and Target Label (y)
features = ['Thickness_Variance_mm', 'Surface_Roughness_Ra', 'Tensile_Strength_MPa']
target = 'Is_Defective'
X = df[features]
y = df[target]

# Step 3: Initialize and Train Logistic Regression Model
model = LogisticRegression(random_state=42)
model.fit(X, y)

# Step 4: Evaluate Model Performance on Dataset
y_pred = model.predict(X)
print("\n--- Model Performance Evaluation ---")
print(f"Accuracy Score: {accuracy_score(y, y_pred) * 100:.2f}%")
print("\nConfusion Matrix:\n", confusion_matrix(y, y_pred))
print("\nClassification Report:\n", classification_report(y, y_pred))

# Step 5: Make Real-time Predictions on Unseen Test Samples
new_samples = pd.DataFrame([[0.02, 1.3, 460], [0.21, 5.2, 295]], columns=features)
predictions = model.predict(new_samples)
probabilities = model.predict_proba(new_samples)

print("--- Real-World Sample Inference ---")
sample_descriptions = ['Component A (Variance: 0.02mm, Roughness: 1.3µm, Tensile: 460 MPa)', 'Component B (Variance:
0.21mm, Roughness: 5.2µm, Tensile: 295 MPa)']
for desc, pred, prob in zip(sample_descriptions, predictions, probabilities):
    print(f"Sample: {desc}")
    print(f"  -> Predicted Class: {pred} | Probability [Class 0, Class 1]: {prob.round(3)}\n")
```

Execution Result & Output Interpretation

```
--- MODEL EVALUATION METRICS ---
Training Accuracy: 100.0%
Model Intercept (β0): 47.4372
Feature Coefficients (β):
```

Vansh Prajapati

- Thickness_Variance_mm: -0.0002
- Surface_Roughness_Ra: 0.0035
- Tensile_Strength_MPa: -0.1250

Confusion Matrix:
[[10 0]
[0 10]]

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	10
accuracy			1.00	20
macro avg	1.00	1.00	1.00	20
weighted avg	1.00	1.00	1.00	20

--- NEW SAMPLE PREDICTIONS ---
[Component A (Variance: 0.02mm, Roughness: 1.3µm, Tensile: 460 MPa)]
-> Predicted: Class 0 (Prob 0: 100.0%, Prob 1: 0.0%)
[Component B (Variance: 0.21mm, Roughness: 5.2µm, Tensile: 295 MPa)]
-> Predicted: Class 1 (Prob 0: 0.0%, Prob 1: 100.0%)

Vansh Prajapati